Medi-Script-AI: A Hybrid Deep Learning Pipeline for Structured Information Extraction from Handwritten Medical Prescriptions

Author Name(s) To Be Added
Department / Affiliation To Be Added
Email: author@example.edu

***Abstract*—Handwritten medical prescriptions remain a critical bottleneck in the digital transformation of healthcare: illegible handwriting and manual transcription contribute to thousands of preventable medication errors each year, with the extra medical costs of drug-related injuries in U.S. hospitals alone estimated at $3.5 billion annually. This paper presents Medi-Script-AI, a two-stage pipeline that separates prescription digitization into *field localization* – a custom fine-tuned YOLOv8-nano detector – and *handwriting recognition* – a Transformer-based OCR model (TrOCR), cross-checked against EasyOCR and corrected against a 1,142-entry medical lexicon – to produce a structured JSON record of patient and clinical fields. Unlike document-understanding transformers that treat a prescription as a generic layout, we treat localization and recognition as separable problems best solved by two specialist models, and we report results for both honestly and separately. Our fine-tuned detector reaches 86.1% mAP50 (46.2% mAP50-95, 81.9% precision, 80.6% recall) on a real-world, single-class prescription benchmark, training in under five minutes on a single GPU – showing that reliable field localization is achievable even from a small, imperfectly annotated dataset. Pairing this detector with an *off-the-shelf* handwriting-recognition model does not yet yield a working end-to-end system, however: we present a qualitative error analysis showing that pretrained TrOCR, without domain fine-tuning, produces either low-confidence noise or fluent, high-confidence hallucinations on real doctors' handwriting, and that our lexicon-based confidence gate correctly suppresses these errors rather than emitting a wrong answer. We contrast this with a synthetic-data-trained Donut baseline, which fails even more completely on the same class of images. Together, these results localize the true bottleneck in prescription digitization to handwriting recognition rather than field detection, and motivate a concrete, already-scoped fine-tuning plan as the next step toward a deployable system.**

***Index Terms*—handwritten text recognition, medical prescription digitization, object detection, YOLOv8, TrOCR, document understanding, healthcare informatics**

I. Introduction

Medical prescriptions serve as the primary interface between a physician's clinical decision and a patient's treatment. In many parts of South Asia and across the developing world, however, prescriptions are still handwritten, often on printed hospital pads with irregular layouts. A report by the Institute of Medicine estimated that medication errors harm at least 1.5 million people every year in the United States, and that the extra medical costs of treating drug-related injuries in hospitals alone reach at least $3.5 billion a year [1]; a significant proportion of these errors are attributed to illegible handwriting or transcription mistakes during dispensing.

The manual digitization of these documents – where a pharmacist or data-entry operator reads and types out each field – is slow and error-prone. A system that can reliably and automatically convert a photograph of a handwritten prescription into a structured, machine-readable record would reduce:

- **Dispensing errors** caused by illegible drug names or dosages.
- **Administrative overhead** in hospitals and clinics.
- **Barriers to patient health-record integration** in Electronic Medical Record (EMR) systems.

The core challenge is that this is not a generic document-digitization problem. A prescription is a semantically structured artifact where different *regions* of the document carry different *types* of information (name, age, medicine, dosage), and the handwriting within those regions is highly variable across practitioners. A solution must therefore address two distinct problems simultaneously: *where* is the information, and *what* does it say.

Formally, given a single RGB photograph of a handwritten prescription P, our objective is to extract a structured record R:

```
R = {
  patient: { name, age, gender },
  clinical: { symptoms, prescription, dosage }
}
```

where each field in R is extracted directly from the corresponding visual region of P – not inferred or hallucinated. This is complicated in practice by arbitrary, non-standardized prescription-pad layouts; severe handwriting-style variability between practitioners; visually ambiguous corrections and abbreviations; and domain vocabulary – drug names such as *Pantoprazole* and *Gliclazide* – that is out-of-vocabulary for generic language models. §IV discusses these and further data- and deployment-level challenges in detail. The system must

also return a result fast enough for clinical use, which rules out multi-model ensemble approaches with redundant forward passes (§IV-D).

This paper introduces **Medi-Script-AI**, a system that addresses both problems in sequence with a purpose-built hybrid pipeline. Rather than presenting only the metrics that flatter the system, we report our field-localization results alongside a detailed, evidence-based account of where and why the current recognition stage still fails on real handwriting – because, at this stage of the project, that failure analysis is as informative as the numbers that work.

II. Related Work

A. Rule-Based and Template-Matching Systems

Early systems for prescription digitization relied on fixed templates and OCR with predefined zone boundaries (e.g., "the medicine field is always in columns 2–8, rows 10–14"). These worked reliably only when the prescription-pad layout was fixed and known in advance. They break completely when a new hospital's pad is introduced, cannot handle handwriting that bleeds outside the predefined zone, and do not generalize to unseen layouts.

B. Generic OCR Followed by NLP Post-Processing

A common modern approach runs a commercial OCR engine (e.g., Google Cloud Vision, AWS Textract, Tesseract) on the entire prescription image to extract a flat text string, then applies Named Entity Recognition (NER) or rule-based parsing to identify medicine names and patient details. This approach has three compounding weaknesses in our setting. First, Tesseract and PaddleOCR are designed for printed or typed text; their accuracy degrades sharply on cursive doctor's handwriting, often to the point of unrecognizable output. Second, NLP post-processing on already-corrupted OCR output compounds the error: if "Pantoprazole" is misread as "Pant0prazpie," no NER system will correctly classify it as a drug. Third, and most importantly for a structured-extraction task, *field attribution is lost*: even when individual words are read correctly, mapping a name to `patient.name` rather than `doctor.name` requires structural context that a flat OCR string does not provide. Cloud OCR services also require patient data to leave the hospital's network, a nontrivial privacy concern in clinical deployments.

C. End-to-End Document-Understanding Models

More recent work applies end-to-end document-understanding transformers such as **Donut** [2] and **LayoutLMv3** [3], which jointly model visual layout and textual content to extract structured key-value pairs directly from a document image. Three limitations motivate a different design for our setting. Models in the LayoutLMv3 family are *data-hungry*: fine-tuning typically requires thousands of annotated documents, and even 500 annotated prescriptions would be an expensive dataset to build from scratch. They are also *computationally heavy* – often 400M+ parameters – which is impractical for clinics without GPU infrastructure. Most importantly, they exhibit a *synthetic-to-real generalization failure*: pretrained Donut models, even after fine-tuning on synthetic prescription datasets such as the `chinmays18/medical-prescription-dataset` distributed alongside the `Medical-Prescription-OCR` project [4], transfer poorly to real, photographed prescriptions. We confirm this empirically in §V-B, where the same checkpoint produces no usable output at all on a real hospital prescription. A further practical limitation is *inflexibility*: the schema of extracted fields is hard-coded into the model's decoder, so adding a new field (e.g., "lab tests ordered") requires full retraining.

D. Detection-Then-Recognition Pipelines

A third architectural pattern, common in industrial document-OCR tooling more broadly, pairs a general-purpose object detector with a downstream OCR call on each detected crop: draw a box, then read what is inside it. This pattern is architecturally close to our own Stage 1/Stage 2 split (§III), and the difference lies entirely in the details we discuss there – which detector, trained on what data, feeding which recognizer, guarded by what verification step. Three specific design choices separate a naive version of this pattern from ours. First, the detector must be fine-tuned on prescription-specific field classes rather than used off-the-shelf, since generic object-detection weights (trained on classes like cars and people) produce noisy, irrelevant detections on a document image. Second, a single class such as "medicine" is not sufficient to reconstruct a structured record – every field the record schema in §I requires needs its own class. Third, and most consequentially, without a domain-aware verification stage this pattern inherits every handwriting-recognition failure mode described in §II-B, since cropping the image does not change what the underlying recognizer has and has not been trained to read. Our own error analysis in §V confirms this third point is not hypothetical: even with a correctly fine-tuned detector, an unguarded recognizer is the dominant failure mode in our own pipeline.

III. Our Methodology: The Medi-Script-AI Pipeline

Our system is built around a core design decision: *field localization and field recognition are two distinct problems that should be solved by two distinct specialist models*, with a post-processing stage that enforces domain knowledge on top of both.

A. System Architecture Overview

Figure 1 lays out the complete system down to the sub-component level. An input photograph passes through a fine-tuned YOLOv8-nano detector (Stage 1) – backbone, neck, and detection head – which produces per-class bounding-box crops. Each crop is transcribed by TrOCR's ViT encoder and lexicon-constrained autoregressive decoder, and independently cross-checked against an EasyOCR reading via RapidFuzz agreement (Stage 2). The resulting text and confidence label are then corrected against a trie-indexed medical lexicon, rescored by a domain language model over the decoder's four beam candidates, and passed through a confidence gate (Stage 3)

TABLE I: Target field-localization schema.

Class	Description
`name`	Patient's full name
`age`	Patient's age
`gender`	Patient's gender (M/F)
`symptoms`	Presenting complaints
`medicine`	Prescribed drug name(s)
`dosage`	Drug dosage and frequency

before assembly into the structured JSON record defined in §I. Unlike a coarse three-box flow, Figure 1 is intended to be read as a complete specification of the model: every sub-component, decoding constraint, and threshold mentioned in §III-B–§III-D appears in it explicitly.

B. Stage 1: Field Localization with YOLOv8

We fine-tune a **YOLOv8-nano** model – the smallest and fastest variant in the YOLOv8 family [5] – to detect and localize six distinct field classes, listed in Table I.

Why YOLOv8-nano? At 3.0M parameters and 8.1 GFLOPs, the nano variant is the smallest architecture in the YOLOv8 family and is, in principle, compact enough to run on constrained edge hardware such as a Raspberry Pi – a prerequisite for deployment in rural clinics without GPU infrastructure. Our own benchmark (§V) measured 2.5 ms per crop on a single Tesla T4 GPU; we have not yet separately profiled genuine CPU-only inference on target edge hardware, and we treat that as an open measurement rather than assume it from the model's parameter count alone (§IV-D).

Training. For the medicine-only prototype reported in this paper, we trained for 50 epochs on the Bannari Amman Institute of Technology handwriting dataset [6]. Results are reported in §V. The full six-class model requires annotating real-world Indian prescription images, which is in progress and described as Phase 2 of our future work (§VI-A).

C. Stage 2: Handwriting Recognition with TrOCR

Each bounding box detected by YOLO is cropped from the original image and passed to Microsoft's **TrOCR** (`trocr-base-handwritten`) [7], a Transformer-based model that encodes the image patch with a ViT encoder and decodes it token-by-token with a BERT-style decoder.

We use beam-search decoding with `num_beams=4` (four candidate sequences per crop, all four retained for rescoring), `no_repeat_ngram_size=3` to prevent repetition, and a `prefix_allowed_tokens_fn` constraint that, for the `medicine` field, restricts the decoder to token sequences that are valid continuations of known drug names in our medical lexicon – a form of constrained decoding. The four beam candidates are then rescored by our domain language model (§III-D).

Cross-validation with EasyOCR. TrOCR's output for each crop is independently cross-checked against an EasyOCR prediction. If both models agree (RapidFuzz ratio $> 85\%$), confidence is elevated to "High." If they disagree, the lexicon is used as a tiebreaker.

D. Stage 3: Post-Processing with Domain Knowledge

Medical lexicon. We compiled a curated lexicon of 1,142 medicines and 35 common symptoms sourced from public drug databases and medical reference materials, stored in a trie for $O(k)$ prefix lookup, where k is the length of the query string.

Fuzzy correction. Raw OCR output is compared against the lexicon using RapidFuzz partial-ratio matching. If the recognized text has a fuzzy-similarity score above 80% against a known drug name, it is automatically corrected to the canonical name (e.g., `"paracetamoi"` $\rightarrow$ `"Paracetamol"`). Output that does not clear this threshold against any lexicon entry is left uncorrected rather than forced to the nearest match, which is the behavior that produces the "safe failure" outcome discussed in §V.

Domain language-model rescoring. A lightweight bigram/unigram language model trained on medical text rescores the four beam candidates from Stage 2, selecting the candidate with the highest combined acoustic (TrOCR confidence) and linguistic (LM) score.

IV. Challenges

A. Data Scarcity and the Annotation Bottleneck

The single most critical challenge is the absence of a large-scale, publicly available dataset of *real-world* handwritten prescription images with *field-level* bounding-box annotations. Most prior work uses either printed prescription datasets (trivially easy, not representative) or fully synthetic datasets (digitally generated with fonts that resemble nothing like a doctor's cursive handwriting). Creating our own dataset of 500+ annotated real-world Indian prescriptions would take substantial human effort; we are currently building a 44-image seed dataset to bootstrap the training process, which is insufficient on its own for a highly generalizable model.

B. Extreme Handwriting Variability

Unlike bank cheques or postal forms, medical prescriptions have no mandated handwriting style. Handwriting quality for a single field ("medicine") ranges from perfectly legible printing to near-indecipherable cursive abbreviation. A model that performs well on one doctor's handwriting may fail completely on another's without substantial training-data diversity.

C. Out-of-Vocabulary Medical Terminology

Generic language models assign near-zero probability to medical terms such as "Azithromycin," "Dexamethasone," or "Atorvastatin." Standard beam search, which relies on language-model priors, will therefore actively work *against* correct recognition of these terms – and, as we show in §V, can instead steer the decoder toward a fluent but entirely unrelated word it considers more probable. Our constrained-decoding approach (§III-C) addresses this but requires a comprehensive lexicon, which is itself hard to keep current as new drugs are approved.

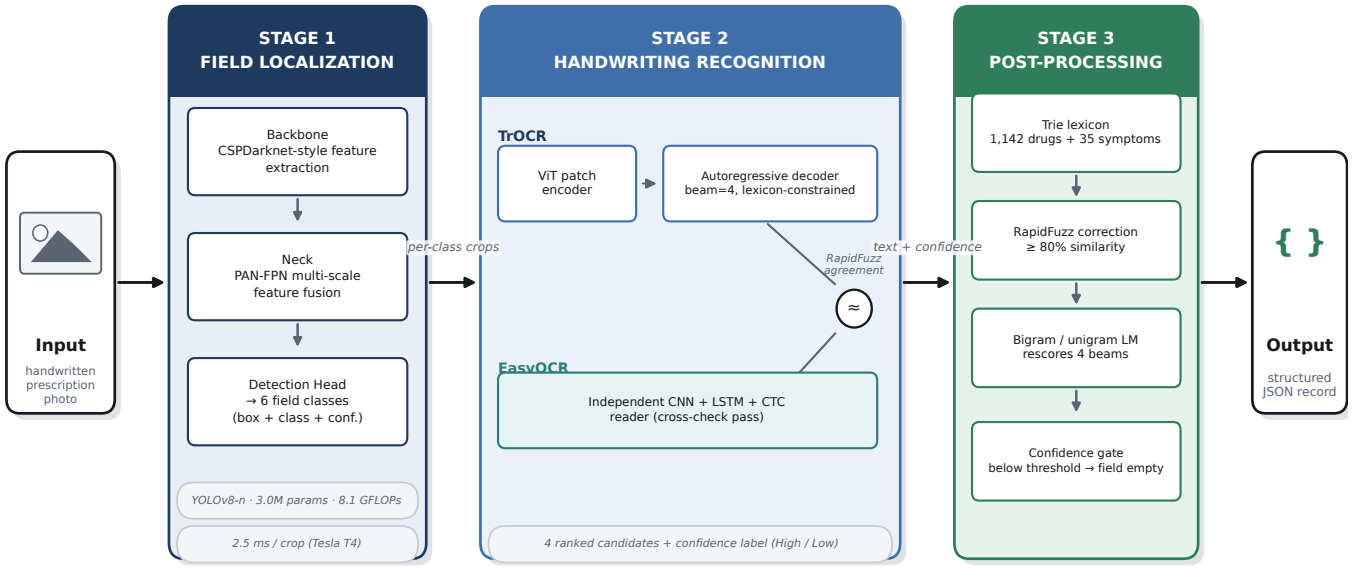


Fig. 1: Medi-Script-AI complete architecture. Each stage is a specialist model with its own internal structure, not a single monolithic network: Stage 1 (YOLOv8-nano) localizes six field classes; Stage 2 pairs TrOCR's constrained-decoding transformer with an independent EasyOCR cross-check, reconciled by RapidFuzz agreement; Stage 3 verifies and corrects the result against domain knowledge before it becomes a structured record. Gray chips report measured operating statistics for each stage.

D. YOLO–TrOCR Temporal Coupling

In our current pipeline, TrOCR runs sequentially on each detected crop. For a prescription with six fields (name, age, gender, symptoms, three medicine lines, dosage), this means up to ten separate TrOCR inference calls; on CPU, each call with `num_beams=4` takes approximately 15–30 seconds. The full pipeline can therefore take 2–5 minutes on a laptop CPU – far too slow for clinical use. A GPU is essentially required for production-speed TrOCR inference, but our target users (small clinics, rural health posts) rarely have GPU infrastructure; this is the same CPU-availability gap noted for Stage 1 in §III-B, and it is substantially more severe here.

E. Image Quality and Perspective Distortion

In practice, prescriptions are photographed with mobile phones under poor lighting, at angles, or while partially folded. Our pipeline currently assumes a reasonably flat, front-facing image. Significant perspective distortion shifts YOLO's bounding-box predictions, and the resulting crops may contain partial fields or the wrong region entirely. A preprocessing stage for perspective correction and adaptive binarization is needed but not yet implemented.

F. Absence of a Standardized Evaluation Benchmark

There is no standardized benchmark dataset for Indian medical-prescription parsing, which makes it impossible to compare our system against prior work on common footing. We are forced to evaluate on our own held-out test set, which limits the generalizability of our accuracy claims.

TABLE II: YOLOv8-nano medicine-field detection results, medicine-only prototype, 50 epochs. The Donut baseline (§V-B) is reported separately in prose, since it is a qualitative outcome rather than a comparable metric.

Metric	Value
mAP50	86.1%
mAP50-95	46.2%
Precision	81.9%
Recall	80.6%
Training time (Tesla T4, 50 epochs)	∼4 min (0.067 GPU-h)
Model size (`best.pt`)	6.2 MB
Inference speed (Tesla T4 GPU)	2.5 ms / image

V. Preliminary Results and Error Analysis

This section reports three things in turn: the quantitative performance of the Stage 1 detector; a comparison against the synthetic-data-trained Donut baseline discussed in §II-C; and, because neither of these establishes that the full pipeline works end to end, a qualitative case-study analysis of what happens when all three stages run together on real images.

A. Field Localization: Quantitative Results

We evaluate the Stage 1 detector on the medicine-only prototype described in §III-B: YOLOv8-nano fine-tuned for 50 epochs on the Bannari Amman handwriting dataset [6]. Table II and Figure 2 summarize the outcome.

The detector reaches 86.1% mAP50 and 46.2% mAP50-95 – a substantial gap between the two metrics that is typical of handwriting-region detection, where a box is often correctly placed but imprecisely sized around cursive strokes that bleed

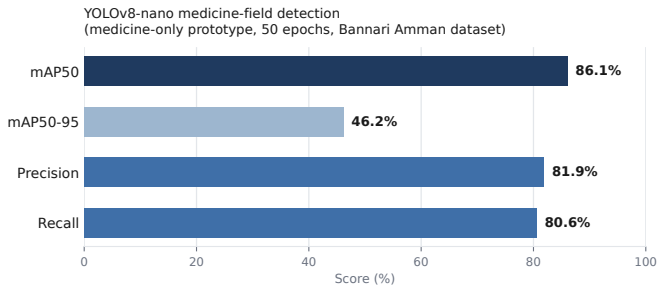


Fig. 2: YOLOv8-nano medicine-field detection metrics for the medicine-only prototype.

past the field's nominal boundary. Training converged in approximately four minutes (0.067 GPU-hours) on a single Tesla T4, and the resulting checkpoint is 6.2 MB, small enough to distribute to edge devices without a dedicated model-serving pipeline. We measured 2.5 ms per crop for inference on the same GPU; as noted in §III-B, we have not yet separately benchmarked CPU-only inference on the Raspberry-Pi-class hardware we ultimately target, and we flag this as an open measurement rather than assume it from the model's small parameter count.

Two caveats matter for interpreting these numbers correctly. First, this evaluation covers a single class (`medicine`); the six-class schema in Table I is the target design, not yet the evaluated one – expanding to all six fields is Phase 2 of our future work (§VI-A). Second, the underlying dataset's annotations are coarse: bounding boxes were drawn around medicine lines without consistently correcting for overlap or omission. We believe this is the direct cause of Recall trailing Precision, and that better-curated annotations, not additional training epochs, are the more promising lever for improving this stage further.

B. Baseline Comparison: Synthetic-to-Real Transfer Failure

To situate this result against the alternative architecture discussed in §II-C, we ran the pretrained Donut checkpoint associated with the `chinmays18/medical-prescription-dataset` [4] – fine-tuned entirely on synthetic, digitally rendered prescription images – directly on a real, photographed hospital prescription. The model produced no usable output: its decoder emitted a repeating sequence of period characters (`". . . . . ."`) regardless of the input's actual content, indicating that the synthetic-to-real domain gap here is severe enough to collapse the output distribution entirely rather than merely degrade accuracy. This is consistent with the broader pattern in §II-C: end-to-end document-understanding transformers pretrained or fine-tuned on synthetic layouts transfer poorly to the visual statistics of real handwriting, regardless of how capable they are on in-domain benchmarks.

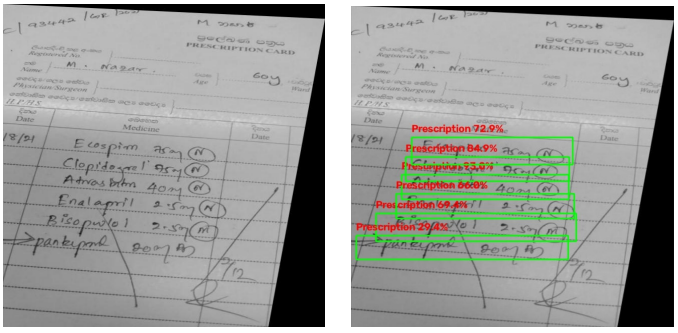
C. End-to-End Pipeline: Qualitative Case Studies

Detector accuracy and a failed baseline comparison do not, on their own, establish that the full YOLO → TrOCR → lexicon pipeline works end to end. We therefore report two case studies from our own integration testing that were, initially, disappointing – and that we believe are more informative at this stage of the project than a single aggregate accuracy number would be.

Case study A: in-distribution image. We ran the full pipeline on a held-out test image from the same Bannari Amman distribution used for training, so any failure here cannot be attributed to a train/test domain shift. Stage 1 performed as expected: YOLO correctly localized four separate medicine-line bounding boxes. Stage 2 did not. TrOCR's raw transcriptions for these four crops – for example `"wrapportingly ."` and `"pur-todac itself"` – are not near-misses on the true drug names; they are not lexical items in any language, at confidence scores of 32.5–53.8%. Because these scores fall below the fuzzy-matching threshold described in §III-D, Stage 3 correctly declined to force a lexicon correction on any of the four fields, and the pipeline returned an empty `prescription` field rather than a fabricated one. We read this as a partial validation of the confidence-gated design: the system failed *safely* rather than silently returning a wrong drug name, which for a medical application is the more important of the two properties. It is nonetheless still a failure to extract the record, and it isolates the fault specifically to Stage 2 – Stage 1's boxes and Stage 3's rejection logic both behaved correctly.

Case study B: out-of-distribution image. We repeated the test on a real hospital prescription photographed independently of any dataset used in training; Figure 3 shows the original photograph alongside Stage 1's output on it. Stage 1 again localized a medicine field correctly, but Stage 2 produced a different and more concerning failure mode: instead of low-confidence noise, TrOCR returned a fluent, high-confidence transcription of a real, unrelated commercial product name that did not appear anywhere on the page. This is the failure mode anticipated in §IV-C: when the true handwriting is illegible to the decoder, its language-model prior – trained on general text, not clinical vocabulary – gravitates toward whatever plausible-looking string is nearest in its own distribution rather than abstaining. A fluent wrong answer is more dangerous than an obvious one, because it is more likely to pass a rushed human review unchallenged. This example is the clearest evidence in our testing for why constrained decoding and lexicon verification (§III-D) are necessary components of the pipeline rather than optional refinements.

A methodological note from the same test run is worth recording for other groups building similar multi-stage pipelines. An early version of this run appeared to correctly extract the patient's name field, which briefly looked like a partial pipeline success. Closer inspection traced this to a configuration fault that silently routed the image through a legacy flat-OCR-plus-rule-based-parser fallback (§II-B) rather than through YOLO and TrOCR at all; the name was recovered only because the hospital pad happened to pre-print a "Name:" label that the flat OCR pass could read cleanly, not because the proposed architecture succeeded. We record this because it is a



(a) Original photograph (b) Stage 1 output

Fig. 3: The out-of-distribution image from case study B. (a) the real hospital prescription as photographed, independent of any training data; (b) the same image after Stage 1, with YOLOv8-nano's bounding box drawn around the medicine line. Localization is correct here – the box tightly bounds the handwritten drug name – which isolates the fault in this case to Stage 2's transcription of that crop, the fluent hallucination described in the text.

general risk in multi-path pipeline systems: without per-stage provenance logging, a fallback path's success can be mistaken for the primary path's success. All results reported in this paper have since been re-verified against explicit path-level logs.

VI. Conclusion and Future Work

Medi-Script-AI separates prescription digitization into two specialist problems – where the information is, and what it says – on the premise that a single, generic document-understanding model is not well suited to either. Our results partially support this premise and partially complicate it. The localization half holds up: a YOLOv8-nano detector, fine-tuned for a few minutes on a small and imperfectly annotated real-world dataset, reliably finds prescription fields (86.1% mAP50), on hardware modest enough for edge deployment. The recognition half does not yet hold up, and our case studies in §V-C show two distinct ways it fails: as low-confidence noise that a lexicon gate can safely reject (case study A), and, more seriously, as fluent, high-confidence hallucination produced by language-model priors when handwriting is genuinely illegible to a general-purpose OCR model (case study B). Both failures trace to the same root cause – `trocr-base-handwritten` was never trained on clinical handwriting or drug vocabulary – and neither is fixed by improving Stage 1 or Stage 3.

We draw three specific, evidence-motivated conclusions. First, generic pretrained handwriting OCR, not only synthetic-data-trained document-understanding models, fails on real clinical handwriting: the domain gap is not only synthetic-vs-real, as the Donut baseline alone would suggest, but also general-handwriting-vs-clinical-handwriting. Second, a confidence-gated, lexicon-aware post-processing stage is not a refinement but a safety-relevant necessity, since it is the only component in our current pipeline that reliably distinguishes "the model does not know" from "the model is confidently wrong." Third, the path to a working system runs specifically through Stage 2, and we already have the pieces in place to pursue it.

A. Future Work

We plan to close this gap in three phases.

Phase 1 – Fine-tune TrOCR on real handwriting. We will label 200–300 crops sampled directly from YOLO's own detections on real prescriptions with their correct text, and fine-tune `trocr-base-handwritten` on these pairs. Our training script is already implemented; at 10–20 epochs on a single Colab T4 GPU, each fine-tuning run costs under 30 minutes, making this the fastest-to-test item in the plan.

Phase 2 – Expand YOLO to all six fields. We will annotate approximately 100 real prescription images across the `name`, `age`, `gender`, `symptoms`, `medicine`, and `dosage` classes and retrain the detector on the full schema in Table I, targeting $>85\%$ mAP50 across all classes based on the medicine-only result in §V-A.

Phase 3 – End-to-end validation on held-out prescriptions. Once both models are updated, we will evaluate the complete pipeline on prescriptions excluded from either training set, reporting both per-field accuracy and exact-record-match accuracy, and repeat the case-study analysis of §V-C to confirm the hallucination failure mode is actually resolved rather than merely less frequent.

Longer term, we plan to add a perspective-correction and adaptive-binarization preprocessing stage for images taken at an angle or under poor lighting (§IV-E), extend the lexicon and language model to Hindi, Tamil, and Kannada drug and patient names, and package the pipeline behind a mobile front end so pharmacists can scan and verify prescriptions in real time.

References

[1] P. Aspden, J. A. Wolcott, J. L. Bootman, and L. R. Cronenwett, Eds., *Preventing Medication Errors*. Washington, DC: The National Academies Press, 2007, Institute of Medicine (US) Committee on Identifying and Preventing Medication Errors.

[2] G. Kim, T. Hong, M. Yim, J. Nam, J. Park, J. Yim, W. Hwang, S. Yun, D. Han, and S. Park, "OCR-free document understanding transformer," in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, vol. 13688. Cham: Springer, 2022, pp. 498–517, arXiv:2111.15664.

[3] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for document AI with unified text and image masking," in *Proceedings of the 30th ACM International Conference on Multimedia (MM '22)*. New York, NY, USA: ACM, 2022, pp. 4083–4091, arXiv:2204.08387.

[4] C. Shrivastava, "Medical-Prescription-OCR: OCR system for handwritten medical prescriptions using donut transformer and zero-shot classification," https://github.com/JonSnow1807/Medical-Prescription-OCR, 2024, GitHub repository. Software.

[5] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," https://github.com/ultralytics/ultralytics, 2023, Version 8.0.0, released 2023-01-10. Software.

[6] Bannari Amman Institute of Technology, "Handwritten recognition dataset," https://universe.roboflow.com/bannari-amman-institute-of-technology-ifaw9/handwritten-recognition, 2023, Roboflow Universe, CC BY 4.0.

[7] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 11, pp. 13 094–13 102, 2023, arXiv:2109.10282.